

Expense Tracker - Project Report

A command-line program that records and summarises personal spending

EXAMPLE REPORT. This shows the sections and the level of detail the Capstone agent expects. Your own .docx report needs exactly three sections: **Problem Statement**, **Approach** and **Conclusion**. Do not paste your code or screenshots into it - your code and your output screenshots both go in the .zip (the screenshots in its **output_screenshots** folder). Write about your own project; do not copy this one.

Problem Statement

People lose track of small everyday spending, so at the end of the month they cannot say where their money went. Paper notes get lost and spreadsheets are too much effort for a ten-second purchase. This project builds a small command-line program that records each expense with an amount, a category and a note, saves it to a file, and shows how much was spent in each category.

The program has to meet these requirements:

- Add an expense with an amount, a category and a note.
- Reject invalid input, such as a zero or negative amount or an unknown category.
- Keep the data between runs, so closing the program loses nothing.
- Show the total spent in each category.
- Include automated tests for the rules.

Approach

The program is split into three modules so that each one has a single job. This keeps the rules easy to test and easy to change without touching the file handling or the command line.

- **tracker.py** holds the rules. It validates an expense and computes category and monthly totals. It never touches the disk, which makes it simple to test.
- **storage.py** reads and writes the JSON file that holds the expenses.
- **main.py** is the command-line interface, built with argparse, that connects the two.

Design decisions

JSON was chosen over a database because the amount of data is small and a plain file needs no setup. Amounts are rounded to two decimals and stored as numbers so totals add up exactly. The category must come from a fixed list, so the totals are not split across spellings such as *food* and *Food*. Validation lives in one place and runs before anything is saved, so a bad value can never reach the file.

How the requirements are met

- Adding an expense goes through *add_expense*, which checks the amount and the category first.
- Invalid input is rejected with a clear message and nothing is written.
- The expense list is loaded at start-up and saved after every change.
- The summary groups expenses by category and prints each total with a simple text bar chart.
- The tests cover valid and invalid expenses, the category totals and the monthly total.

Conclusion

The program meets all five requirements. It records and validates expenses, keeps them between runs, summarises spending by category and is covered by seven automated tests. Keeping the rules separate from the file handling was the most useful decision: it made the code simple to test and simple to change.

What I learned

Deciding what counts as invalid input before writing any code saved time later, and small modules with one job each are much easier to check than one long script.

Limitations and possible improvements

- One user only, and saved expenses cannot yet be edited or deleted.
- Filtering by date range and exporting to CSV would be natural next steps.
- A simple chart of monthly spending would make the summary easier to read.